

1. Linear Search Bug

Given Code

```
int linearSearch(int arr[], int n, int key)
{
    for(int i = 0; i <= n; i++)
    {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
```

Error:

Loop condition is incorrect

Reason:

- Array index starts from 0 and ends at n-1.
- `i <= n` accesses `arr[n]`, causing **array out of bounds error**.

Correct Code

```
int linearSearch(int arr[], int n, int key)
{
    for(int i = 0; i < n; i++)
    {
        if(arr[i] == key)
            return i;
    }

    return -1;
}
```

2. Linear Search Found Condition

Given Code

```
int found = 0;

for(int i = 0; i < n; i++)
{
    if(arr[i] == key)
        found = 1;
}

if(found == 1)
    return 1;
else
```

```
return -1;
```

Error:

Assignment used instead of comparison

Reason:

- = assigns a value.
- == is used to compare values.

Correct Code

```
int linearSearch(int arr[], int n, int key)
{
    int found = 0;

    for(int i = 0; i < n; i++)
    {
        if(arr[i] == key)
            found = 1;
    }

    if(found == 1)
        return 1;
    else
        return -1;
}
```

3. Binary Search Mid Calculation

Given Code

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n-1;

    while(low <= high)
    {
        int mid = (high - low)/2;

        if(arr[mid] == key)
            return mid;

        else if(arr[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}
```

Error:**Incorrect mid calculation****Reason:**

- $(\text{high} - \text{low})/2$ does not give the correct array position.
- Mid should be calculated using $(\text{low} + \text{high})/2$.

Correct Code

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n-1;

    while(low <= high)
    {
        int mid = (low + high)/2;

        if(arr[mid] == key)
            return mid;

        else if(arr[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}
```

4. Binary Search Infinite Loop

Given Code

```
while(low < high)
{
    int mid = (low + high)/2;

    if(arr[mid] == key)
        return mid;

    else if(arr[mid] < key)
        low = mid;

    else
        high = mid;
}
```

Error:**Infinite loop possibility**

Reason:

- low = mid and high = mid may not change values.
- Loop may execute forever.

Correct Code

```
int binarySearch(int arr[], int n, int key)
{
    int low=0, high=n-1;

    while(low <= high)
    {
        int mid=(low+high)/2;

        if(arr[mid]==key)
            return mid;

        else if(arr[mid]<key)
            low=mid+1;

        else
            high=mid-1;
    }

    return -1;
}
```

5. Binary Search Array

Given Code

```
int arr[] = {10,5,20,15,30};

int result = binarySearch(arr,5,15);
```

Error:

Array is not sorted

Reason:

- Binary search works only on sorted data.

Correct Code

```
int arr[] = {5,10,15,20,30};

int result = binarySearch(arr,5,15);
```

6. Stack Push Overflow

Given Code

```
#define MAX 5

int stack[MAX];
int top = -1;

void push(int x)
{
    if(top == MAX)
    {
        printf("Overflow");
        return;
    }

    stack[++top]=x;
}
```

Error:

Overflow condition incorrect

Reason:

- Maximum valid index is MAX-1.

Correct Code

```
#define MAX 5

int stack[MAX];
int top=-1;

void push(int x)
{
    if(top == MAX-1)
    {
        printf("Overflow");
        return;
    }

    stack[++top]=x;
}
```

7. Stack Pop Underflow

Given Code

```
int pop()
{
```

```
if(top == 0)
{
    printf("Underflow");
    return -1;
}

return stack[top--];
}
```

Error:

Underflow condition incorrect

Reason:

- Empty stack condition is `top == -1`.

Correct Code

```
int pop()
{
    if(top == -1)
    {
        printf("Underflow");
        return -1;
    }

    return stack[top--];
}
```

8. Stack Push Increment Error

Given Code

```
void push(int x)
{
    stack[top++] = x;
}
```

Error:

Wrong increment order

Reason:

- Top should increase before inserting the element.

Correct Code

```
void push(int x)
{
```

```
    if(top == MAX-1)
    {
        printf("Overflow");
        return;
    }

    stack[++top]=x;
}
```

9. Stack Initialization

Given Code

```
int stack[5];
int top;

void init()
{
    top=0;
}
```

Error:

Initialization of top is incorrect

Reason:

- Empty stack starts with top = -1.

Correct Code

```
int stack[5];
int top;

void init()
{
    top=-1;
}
```

10. Queue Enqueue Overflow

Given Code

```
#define MAX 5

int queue[MAX];
int front=0,rear=0;

void enqueue(int x)
{
```

```

    if(rear==MAX)
    {
        printf("Overflow");
        return;
    }

    queue[rear++]=x;
}

```

Error:

Overflow condition incorrect

Reason:

- Last valid queue index is MAX-1.
- rear==MAX accesses outside the array.

Correct Code

```

#define MAX 5

int queue[MAX];
int front=0,rear=-1;

void enqueue(int x)
{
    if(rear==MAX-1)
    {
        printf("Overflow");
        return;
    }

    queue[++rear]=x;
}

```

11. Queue Dequeue Operation

Given Code

```

int dequeue()
{
    if(front == rear)
    {
        printf("Underflow");
        return -1;
    }

    return queue[front++];
}

```

Error:

No error

Reason:

- `front == rear` correctly checks whether the queue is empty (for this queue implementation).
- The element is removed and `front` is incremented.

Correct Code

```
int dequeue()
{
    if(front == rear)
    {
        printf("Underflow");
        return -1;
    }

    return queue[front++];
}
```

12. Queue Enqueue Operation

Given Code

```
void enqueue(int x)
{
    queue[rear++] = x;
}
```

Error:**Missing overflow check****Reason:**

- Queue may exceed its maximum size.
- Need to check whether the queue is full before insertion.

Correct Code

```
void enqueue(int x)
{
    if(rear == MAX-1)
    {
        printf("Overflow");
        return;
    }

    queue[rear++] = x;
}
```

13. Circular Queue Enqueue

Given Code

```
void enqueue(int x)
{
    if(rear == MAX-1)
    {
        printf("Overflow");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % MAX;
    queue[rear] = x;
}
```

Error:

Incorrect overflow condition

Reason:

- In circular queue, overflow occurs when:

$(rear + 1) \% MAX == front$

Correct Code

```
void enqueue(int x)
{
    if((rear + 1) % MAX == front)
    {
        printf("Overflow");
        return;
    }

    rear = (rear + 1) % MAX;
    queue[rear] = x;

    if(front == -1)
        front = rear;
}
```

14. Circular Queue Dequeue

Given Code

```
int dequeue()
```

```

{
    if(front == -1)
    {
        printf("Underflow");
        return -1;
    }

    int val = queue[front];

    front = (front + 1) % MAX;

    return val;
}

```

Error:

Queue empty condition is not reset

Reason:

- After deleting the last element, front and rear should become -1.

Correct Code

```

int dequeue()
{
    if(front == -1)
    {
        printf("Underflow");
        return -1;
    }

    int val = queue[front];

    if(front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }

    return val;
}

```

15. Queue Dequeue Modulo Error

Given Code

```

int dequeue()
{
    int val = queue[front];
}

```

```
    if(front == rear)
        front = rear = -1;

    else
        front = front + 1;

    return val;
}
```

Error:

Missing modulo operation

Reason:

- Circular queue requires wrapping of index.

Correct Code

```
int dequeue()
{
    if(front == -1)
    {
        printf("Underflow");
        return -1;
    }

    int val = queue[front];

    if(front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }

    return val;
}
```

16. Infix to Postfix Bracket Handling

Given Code

```
if(ch == ')')
{
    while(stack[top] != '(')
    {
        postfix[j++] = pop();
    }

    pop();
}
```

Error:

No empty stack check

Reason:

- Accessing stack[top] when stack is empty causes error.

Correct Code

```
if(ch == ')')
{
    while(top != -1 && stack[top] != '(')
    {
        postfix[j++] = pop();
    }

    if(top != -1)
        pop();
}
```

17. Operator Precedence

Given Code

```
int precedence(char ch)
{
    if(ch=='+' || ch=='-')
        return 2;

    else if(ch=='*' || ch=='/')
        return 1;

    return 0;
}
```

Error:

Precedence values are incorrect

Reason:

- Multiplication and division have higher precedence than addition and subtraction.

Correct Code

```
int precedence(char ch)
{
    if(ch=='+' || ch=='-')
        return 1;
}
```

```
    else if(ch=='*' || ch=='/')
        return 2;

    return 0;
}
```

18. Postfix Evaluation

Given Code

```
int a = pop();
int b = pop();

int result = a - b;

push(result);
```

Error:

Wrong operand order

Reason:

- In postfix evaluation:
 - First popped value is second operand.
 - Second popped value is first operand.

Correct Code

```
int b = pop();
int a = pop();

int result = a - b;

push(result);
```

19. Balanced Bracket Checking

Given Code

```
if((ch == '[' && stack[top] == '[') ||
    (ch == '}' && stack[top] == '{'))
{
    pop();
}
else
{
    return 0;
}
```

Error:

No empty stack check before accessing stack[top]

Reason:

- If stack is empty, accessing stack[top] causes error.

Correct Code

```
if(top == -1)
{
    return 0;
}

if((ch == ']' && stack[top] == '[') ||
   (ch == '}' && stack[top] == '{'))
{
    pop();
}
else
{
    return 0;
}
```

20. Linked List Traversal and Insertion

Given Code

```
struct node *temp = head;

while(temp != NULL)
{
    temp = temp->next;
}

temp->next = newNode;
```

Error:

temp becomes NULL before insertion

Reason:

- Loop moves temp until it becomes NULL.
- Cannot access temp->next after that.

Correct Code

```
struct node *temp = head;
```

```
while(temp->next != NULL)
{
    temp = temp->next;
}
```

```
temp->next = newNode;
```

21. Stack Push Overflow

Given Pseudocode

```
PUSH(stack, x):
```

```
if top == MAX:
    print("Overflow")
```

```
else:
    top = top + 1
    stack[top] = x
```

Error:

Overflow condition is incorrect

Reason:

- Array index starts from 0.
- The last valid stack position is MAX-1.
- Checking top == MAX causes overflow one position late.

Correct Code

```
#define MAX 5

int stack[MAX];
int top = -1;

void push(int x)
{
    if(top == MAX-1)
    {
        printf("Overflow");
        return;
    }

    top++;
    stack[top] = x;
}
```

22. Queue Enqueue Overflow

Given Code

```
ENQUEUE(Q, x):  
  
if rear == MAX-1:  
    print("Overflow")  
  
else:  
    rear = rear + 1  
    Q[rear] = x
```

Error:

Front is not reset after deletion

Reason:

- In a linear queue, deleted spaces cannot be reused.
- Need circular queue or reset front and rear.

Correct Code

```
void enqueue(int x)  
{  
    if((rear+1)%MAX == front)  
    {  
        printf("Overflow");  
        return;  
    }  
  
    rear = (rear+1)%MAX;  
    queue[rear] = x;  
}
```

23. Linked List Traversal

Given Code

```
temp = head;  
  
while(temp.next != NULL)  
{  
    printf(temp.data);  
    temp = temp.next;  
}
```

Error:

Last node is not printed

Reason:

- Loop stops when temp.next becomes NULL.
- Last node is skipped.

Correct Code

```
void display(struct node *temp)
{
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

24. Binary Search Mid Calculation

Given Code

```
mid = (low + high) / 2;
```

Error:

Integer overflow possibility

Reason:

- When low and high are very large, low + high can exceed integer limit.

Correct Code

```
mid = low + (high - low) / 2;
```

25. Linear Search Loop Boundary

Given Code

```
for(i = 0; i <= n; i++)
{
    if(arr[i] == key)
        return i;
}
```

Error:

Loop boundary error

Reason:

- arr[n] is outside the array.

Correct Code

```
int linearSearch(int arr[], int n, int key)
{
    for(int i=0;i<n;i++)
    {
        if(arr[i]==key)
            return i;
    }

    return -1;
}
```

26. Stack POP Operation

Given Code

```
POP(stack):

if top == 0:
    print("Underflow")

else:
    return stack[top--]
```

Error:

Underflow condition incorrect

Reason:

- Empty stack condition is top == -1.

Correct Code

```
int pop()
{
    if(top == -1)
    {
        printf("Underflow");
        return -1;
    }

    return stack[top--];
}
```

27. Binary Search on Unsorted Data

Error:

Binary search gives incorrect results.

Reason:

- Binary search requires sorted data.

Correct Code

```
#include<stdio.h>

int binarySearch(int arr[], int n, int key)
{
    int low=0, high=n-1;

    while(low<=high)
    {
        int mid=(low+high)/2;

        if(arr[mid]==key)
            return mid;

        else if(arr[mid]<key)
            low=mid+1;

        else
            high=mid-1;
    }

    return -1;
}
```

28. Circular Queue Full Condition

Given Code

```
if(rear == MAX)
```

Error:**Wrong full queue condition****Reason:**

- Circular queue full condition is:

$(rear + 1) \% MAX == front$

Correct Code

```
if((rear+1)%MAX == front)
{
    printf("Queue Full");
    return;
}
```

29. Delete Node

Given Code

```
delete(node)
{
    node = node.next;
}
```

Error:

Node is not actually deleted

Reason:

- Pointer is changed only locally.
- Memory is not released.

Correct Code

```
void deleteNode(struct node *temp)
{
    free(temp);
}
```

30. Time Complexity Analysis

Question:

Identify the truth in complexity analysis.

Error:

No code error.

Correct Explanation:

Binary Search:

- Works only on sorted arrays.
- Worst Case Time Complexity = $O(\log n)$

Stack Operations:

- Push = $O(1)$
- Pop = $O(1)$

Queue Operations:

- Enqueue = $O(1)$
- Dequeue = $O(1)$

Example Code

```
int binarySearch(int arr[], int n, int key)
{
    int low=0, high=n-1;

    while(low<=high)
    {
        int mid=(low+high)/2;

        if(arr[mid]==key)
            return mid;

        else if(arr[mid]<key)
            low=mid+1;

        else
            high=mid-1;
    }

    return -1;
}
```

31. Reverse Linked List

Given Code

```
while(current != NULL)
{
    current->next = prev;
    prev = current;
    current = current->next;
}
```

Error:

Pointer loss

Reason:

- After changing `current->next`, the original next node address is lost.
- Need a temporary pointer to store the next node.

Correct Code

```
struct node* reverse(struct node *head)
{

```

```

struct node *prev = NULL;
struct node *current = head;
struct node *next;

while(current != NULL)
{
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}

return prev;
}

```

32. Insert Node at Beginning

Given Code

```

newNode->next = head;
head = newNode;

```

Error:

No error

Reason:

- This is the correct method to insert a node at the beginning of a linked list.

Correct Code

```

struct node* insertBeginning(struct node *head, int val)
{
    struct node *newNode =
        malloc(sizeof(struct node));

    newNode->data = val;
    newNode->next = head;

    return newNode;
}

```

33. Queue Dequeue Operation

Given Code

```

DEQUEUE(Q):

if front == -1:

```

```
    print("Underflow")

else:
    return Q[front++]
```

Error:

Reset condition missing

Reason:

- After removing the last element, queue should become empty.
- Front and rear must be reset.

Correct Code

```
int dequeue()
{
    if(front == -1)
    {
        printf("Underflow");
        return -1;
    }

    int val = queue[front++];

    if(front > rear)
    {
        front = rear = -1;
    }

    return val;
}
```

34. Stack Peek Operation

Given Code

```
PEEK(stack):
return stack[top+1]
```

Error:

Off-by-one error

Reason:

- Top element is stored at stack[top].
- top+1 accesses an invalid position.

Correct Code


```
int peek()
{
    if(top == -1)
    {
        printf("Stack Empty");
        return -1;
    }

    return stack[top];
}
```

35. Binary Search Loop Condition

Given Code

```
while(low < high)
{
    mid = (low + high)/2;
}
```

Error:

Wrong loop condition

Reason:

- Search may miss the last element.
- Use low <= high.

Correct Code

```
while(low <= high)
{
    int mid = (low + high)/2;

    if(arr[mid] == key)
        return mid;

    else if(arr[mid] < key)
        low = mid + 1;

    else
        high = mid - 1;
}
```

36. Linked List Insert Function

Given Code

```
void insert(struct node *head, int val)
```

```

{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
    temp->next = head;

    head = temp;
}

```

Error:

Head is not updated in main function

Reason:

- Head is passed by value.
- Original head will not change.

Correct Code

```

struct node* insert(struct node *head, int val)
{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
    temp->next = head;

    return temp;
}

```

37. Doubly Linked List Insertion

Given Code

```

newNode->next = head;
head->prev = newNode;
head = newNode;

```

Error:

Missing NULL check

Reason:

- If list is empty, head->prev causes error.

Correct Code

```

newNode->next = head;

```

```
if(head != NULL)
{
    head->prev = newNode;
}

newNode->prev = NULL;
head = newNode;
```

38. Queue Overflow Check

Given Code

```
if(rear == MAX-1)
{
    printf("Full");
}

queue[++rear] = x;
```

Error:

Missing return after overflow

Reason:

- Even after printing "Full", insertion continues.

Correct Code

```
void enqueue(int x)
{
    if(rear == MAX-1)
    {
        printf("Full");
        return;
    }

    queue[++rear] = x;
}
```

39. Linear Search Last Element

Given Code

```
for(i=0; i<n-1; i++)
{
    if(arr[i] == key)
        return i;
}
```

Error:

Last element is not checked

Reason:

- Loop stops at n-2.

Correct Code

```
int linearSearch(int arr[], int n, int key)
{
    for(int i=0;i<n;i++)
    {
        if(arr[i]==key)
            return i;
    }

    return -1;
}
```

40. Binary Search Condition

Given Code

```
while(low < high)
{
    mid = (low + high)/2;
}
```

Error:

Loop condition incorrect

Reason:

- It may skip the last possible element.

Correct Code

```
int binarySearch(int arr[], int n, int key)
{
    int low=0;
    int high=n-1;

    while(low<=high)
    {
        int mid=(low+high)/2;

        if(arr[mid]==key)
            return mid;
    }
}
```

```
        else if(arr[mid]<key)
            low=mid+1;

        else
            high=mid-1;
    }

    return -1;
}
```

41. Searching Purpose

Error:

No code error. It is a concept question.

Explanation:

- Searching is used to **find a specific element in a data structure.**

Correct Code Example

```
int linearSearch(int arr[], int n, int key)
{
    for(int i = 0; i < n; i++)
    {
        if(arr[i] == key)
            return i;
    }

    return -1;
}
```

42. Binary Search Reduction

Error:

No code error.

Explanation:

- Binary search reduces the search space by **half in every step.**

Correct Code

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0;
    int high = n - 1;
```

```

while(low <= high)
{
    int mid = (low + high) / 2;

    if(arr[mid] == key)
        return mid;

    else if(arr[mid] < key)
        low = mid + 1;

    else
        high = mid - 1;
}

return -1;
}

```

43. Infix to Postfix Stack Priority

Given Code

```

while(priority(stack[top]) >= priority(symbol))
{
    printf("%c", pop());
}

```

Error:

No check for empty stack.

Reason:

- Accessing stack[top] when stack is empty causes an error.

Correct Code

```

while(top != -1 &&
    priority(stack[top]) >= priority(symbol))
{
    printf("%c", pop());
}

```

44. Circular Queue Enqueue

Given Code

```

if((rear + 1) % MAX == front)
{
    printf("Full");
}

```

```
}

rear = (rear + 1) % MAX;
queue[rear] = x;
```

Error:

Missing return after full condition.

Reason:

- Program continues insertion even after queue becomes full.

Correct Code

```
void enqueue(int x)
{
    if((rear + 1) % MAX == front)
    {
        printf("Full");
        return;
    }

    rear = (rear + 1) % MAX;
    queue[rear] = x;
}
```

45. Linked List Insert Function

Given Code

```
struct node* insert(struct node *head, int val)
{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
    temp->next = head;
}
```

Error:

Missing return statement.

Correct Code

```
struct node* insert(struct node *head, int val)
{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
```

```
temp->next = head;

return temp;
}
```

46. Linked List Insert Logic

Given Code

```
struct node* insert(struct node *head, int val)
{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
    temp->next = NULL;

    head = temp;

    return head;
}
```

Error:

Wrong pointer assignment.

Reason:

- New node should point to the existing head.

Correct Code

```
struct node* insert(struct node *head, int val)
{
    struct node *temp =
        malloc(sizeof(struct node));

    temp->data = val;
    temp->next = head;

    return temp;
}
```

47. Doubly Linked List Insert End

Given Code

```
while(ptr->next != NULL)
{
    ptr = ptr->next;
}
```



```
}  
  
ptr->next = temp;  
temp->prev = ptr;
```

Error:

Missing return statement.

Correct Code

```
struct node* insertEnd(struct node *head, int val)  
{  
    struct node *temp =  
        malloc(sizeof(struct node));  
  
    temp->data = val;  
    temp->next = NULL;  
  
    if(head == NULL)  
    {  
        temp->prev = NULL;  
        return temp;  
    }  
  
    struct node *ptr = head;  
  
    while(ptr->next != NULL)  
    {  
        ptr = ptr->next;  
    }  
  
    ptr->next = temp;  
    temp->prev = ptr;  
  
    return head;  
}
```

48. Doubly Linked List Previous Pointer

Given Code

```
ptr->next = temp;  
ptr->prev = temp;
```

Error:

Wrong previous pointer assignment.

Reason:

- ptr->prev should not point to the new node.
- New node's previous pointer should point to ptr.

Correct Code

```
ptr->next = temp;  
temp->prev = ptr;  
temp->next = NULL;
```

49. Binary Search Complexity

Error:

No code error.

Explanation:

Binary Search Time Complexity:

Best Case : $O(1)$
Average Case : $O(\log n)$
Worst Case : $O(\log n)$

Correct Code

```
int binarySearch(int arr[], int n, int key)  
{  
    int low = 0;  
    int high = n-1;  
  
    while(low <= high)  
    {  
        int mid = (low + high)/2;  
  
        if(arr[mid] == key)  
            return mid;  
  
        else if(arr[mid] < key)  
            low = mid + 1;  
  
        else  
            high = mid - 1;  
    }  
  
    return -1;  
}
```

50. Circular Queue Traffic Signal

Given Code

```
rear = (rear + 1) % MAX;
```

```
queue[rear] = vehicle;

if(rear == front)
{
    printf("Queue Full");
}
```

Error:

Full condition checked after insertion.

Reason:

- Queue should be checked before inserting a new element.

Correct Code

```
void enqueue(int vehicle)
{
    if((rear + 1) % MAX == front)
    {
        printf("Queue Full");
        return;
    }

    rear = (rear + 1) % MAX;

    queue[rear] = vehicle;
}
```